

Where Things Stand

Use an AI coding agent to autonomously convert ROCKE-3D (NASA GISS's Fortran GCM) to Python/JAX, and measure accuracy, CPU/GPU performance, conversion cost, and gotchas — validated against real production restart data (P2SAoM40).

14 / 17

Modules faithfully ported

Validated directly against real Fortran. 3 remain placeholders — see slide 3.

1e-9 to 1e-3

Accuracy vs. real Fortran

PBL + DRYCNV, same inputs, CPU and GPU — floating-point-level agreement.

2.3–6.3×

GPU speedup vs. real Fortran

Measured, not estimated — first real GPU run, 2026-09-20 (kernel-level).

Mixed on CPU

JAX is not a blanket CPU win

Fortran beats JAX-CPU on DRYCNV. The case for JAX here is GPU, not CPU.

The comparison that matters: Every number here is **JAX vs. the real production Fortran model.**

PERFORMANCE

Real Fortran vs. JAX — CPU and GPU

Kernel-level (mean of 100 calls)

"Kernel-level" = timing one physics routine in isolation (DRYCNV, PBL), not the full pipeline -- isolates the JAX port's raw per-call speed, and is the only comparison with real, measured GPU numbers so far.

DRYCNV



PBL similarity



Same shared inputs on every leg. Grid: P2SAoM40's real 72x46x40.

Full physics chain (CPU + GPU)

PBL + radiation + surface + ground, driven by P2SAoM40's real restart state, chained in the real per-timestep order. Radiation excluded below (simplified graybody stand-in).

5.5x

CPU vs. Fortran (48 vs. 264 ms)

~1.0x

GPU vs. that node's CPU (33.0 vs. 33.5 ms) -- no benefit

Real A100 result (2026-09-22) corrects an earlier ~20-30x GPU *estimate* for this scope, used throughout EXECUTIVE_SUMMARY.md's ROI section (now flagged there). Still ~8x faster than Fortran on GPU -- the grid (3,312 pts) is just too small to benefit from GPU parallelism the way the estimate assumed.

Reading this honestly

JAX-CPU is **not** a blanket win — it loses DRYCNV outright to Fortran on CPU (small-array dispatch overhead dominates at this grid size).

GPU wins decisively on isolated kernels (2.3-6.3x) — but not on the full chained pipeline (~1.0x). The grid is too small either way for GPU to be a clean win by itself.

Recommendation & Open Items

The remaining 3 modules — port or defer?

Why they're still placeholders: the original port deliberately left these three as documented stand-ins (e.g. LAKES' lkmix is a no-op) to reach interface-complete (17/17) first, prioritizing full validation of the columnar physics (PBL, DRYCNV) instead -- a scoping choice, not an oversight.

SEAICE (core thermodynamics) — port next

Physically active in every P2SAoM40-class run, not optional. Dominates polar surface energy balance — exactly where the current 0.987/0.965 global correlation checks are least able to see a local error.

ATURB (PBL-top-finding) — port next

Feeds surface-flux accuracy globally, not just at the poles. Also tests whether the vmap-over-grid-cells approach holds up on harder physics.

LAKES (lkmix) — defer

Small fraction of Earth's surface — lower accuracy payoff for the effort. Revisit only if a lake-focused science need requires it.

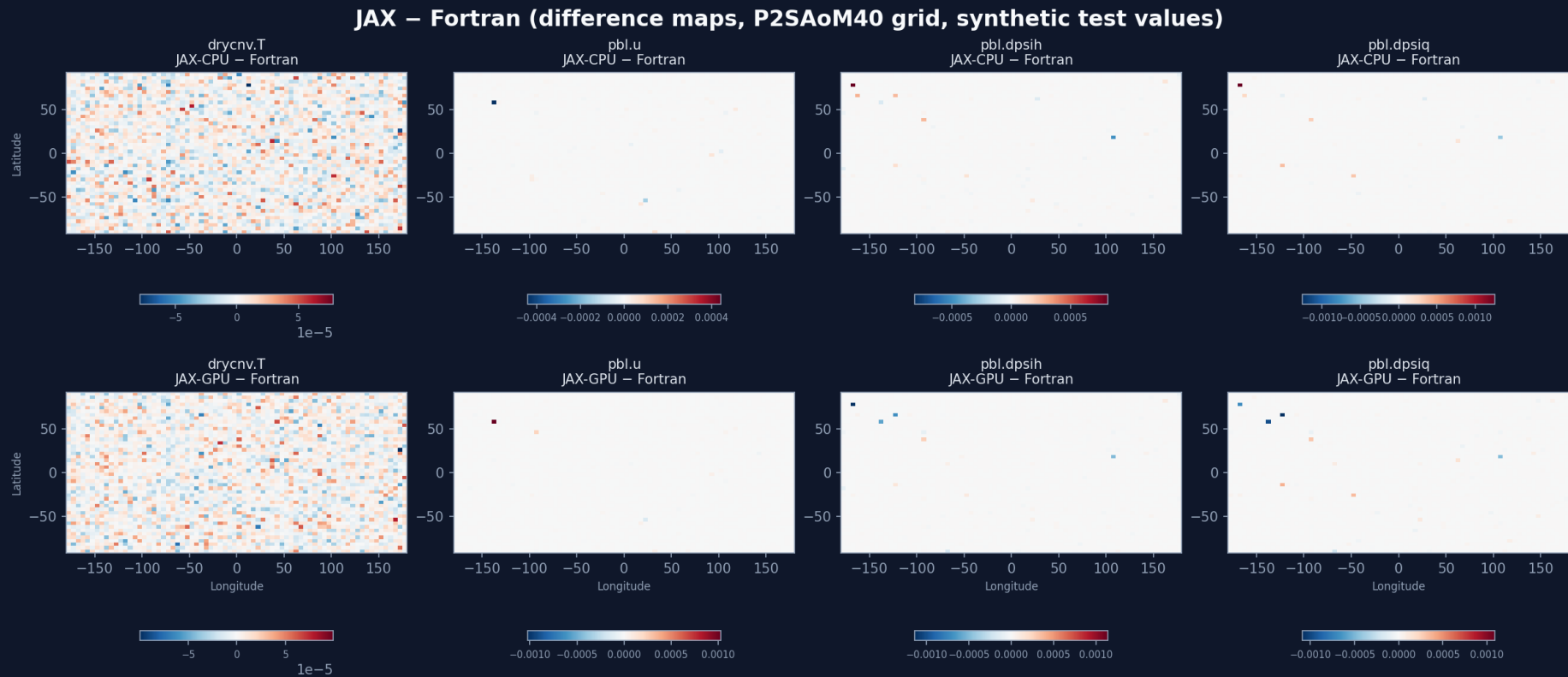
Open items

- 1 Measure the full physics-chain GPU run — **done**, 2026-09-22, real A100: ~1.0x (no benefit). Follow-up: re-derive the ROI figures in EXECUTIVE_SUMMARY.md that assumed the old ~20-30x estimate.
- 2 Fix the wind-speed convention bug in the shared FLUXES/SURFACE module files themselves — currently only worked around locally.
- 3 Port SEAICE + ATURB, then re-run the correlation check — expect it to improve or reveal exactly where it was masking a gap.

OUTPUT MAPS

Where JAX and Fortran Differ, Spatially

Difference maps only, from visualize_p2saom40_kernel_maps.ipynb / outputs/p2saom40_kernel_*_diff_*.html, on P2SAoM40's real 72x46 grid. Field values are synthetic test data, not real climate.



The bright outlier points in pbl.u/dpsih/dpsiq come from a few grid cells with near-zero Monin-Obukhov length -- both Fortran and JAX reproduce the same outliers, so these are still the floating-point-level diffs reported in FINDINGS.md section 2d, not a bug.